From COO to CSR: Counting-Sort Conversion

1 Problem

Let $A \in \mathbb{R}^{M \times N}$ have $\texttt{nnz}$ stored entries given in coordinate (COO) form, e.g. as read from a Matrix Market file after subtracting 1 from every index:

$$(R[k],\, C[k],\, V[k]), \qquad k = 0, \ldots, \texttt{nnz} - 1,$$

in arbitrary order. The goal is to build the compressed sparse row (CSR) arrays

$$\texttt{row_ptr}[0..M], \qquad \texttt{col}[0..\texttt{nnz} - 1], \qquad \texttt{val}[0..\texttt{nnz} - 1],$$

such that the entries of row i occupy positions $\texttt{row_ptr}[i], \ldots, \texttt{row_ptr}[i+1] - 1$. All indices are 0-based.

2 Algorithm

Algorithm 1 COO to CSR (counting sort by row)

Require: M, $\texttt{nnz}$; COO arrays R, C, V of length $\texttt{nnz}$, with $0 \le R[k] < M$
Ensure: CSR arrays $\texttt{row_ptr}, \texttt{col}, \texttt{val}$

allocate $\texttt{row_ptr}[0..M]$, $\texttt{col}[0..\texttt{nnz} - 1]$, $\texttt{val}[0..\texttt{nnz} - 1]$, $\texttt{next}[0..M - 1]$
// Step 1: count nonzeros per row (stored shifted by one)
for $i \leftarrow 0$ **to** M **do**
    $\texttt{row_ptr}[i] \leftarrow 0$
end for
for $k \leftarrow 0$ **to** $\texttt{nnz} - 1$ **do**
    $\texttt{row_ptr}[R[k] + 1] \leftarrow \texttt{row_ptr}[R[k] + 1] + 1$
end for
// Step 2: prefix sum; now $\texttt{row_ptr}[i]$ = start of row i
for $i \leftarrow 1$ **to** M **do**
    $\texttt{row_ptr}[i] \leftarrow \texttt{row_ptr}[i] + \texttt{row_ptr}[i - 1]$
end for
// Step 3: write cursors
for $i \leftarrow 0$ **to** $M - 1$ **do**
    $\texttt{next}[i] \leftarrow \texttt{row_ptr}[i]$
end for
// Step 4: scatter each triplet into its row segment
for $k \leftarrow 0$ **to** $\texttt{nnz} - 1$ **do**
    $r \leftarrow R[k]$
    $p \leftarrow \texttt{next}[r]$
    $\texttt{col}[p] \leftarrow C[k]$
    $\texttt{val}[p] \leftarrow V[k]$
    $\texttt{next}[r] \leftarrow p + 1$
end for
return $\texttt{row_ptr}, \texttt{col}, \texttt{val}$

Remarks.

- After Step 2, $\texttt{row_ptr}[M] = \texttt{nnz}$.
- Step 4 is *stable*: entries of the same row keep their input order. Hence, if the input is already sorted by (R, C), the output has sorted columns and Step 5 below can be skipped.
- The array `next` can be avoided by using `row_ptr` itself as the cursor array and shifting it back by one position at the end ($\texttt{row_ptr}[i] \leftarrow \texttt{row_ptr}[i-1]$ for $i = M, \ldots, 1$; $\texttt{row_ptr}[0] \leftarrow 0$).

Algorithm 2 Optional: sort columns within each row (Step 5)

Require: CSR arrays $\texttt{row_ptr}, \texttt{col}, \texttt{val}$ for M rows
Ensure: Same matrix, with `col` increasing within every row

for $i \leftarrow 0$ **to** $M - 1$ **do**
  // insertion sort on the segment of row i
  for $p \leftarrow \texttt{row_ptr}[i] + 1$ **to** $\texttt{row_ptr}[i+1] - 1$ **do**
    $c \leftarrow \texttt{col}[p], \quad v \leftarrow \texttt{val}[p]$
    $q \leftarrow p - 1$
    while $q \geq \texttt{row_ptr}[i]$ **and** $\texttt{col}[q] > c$ **do**
      $\texttt{col}[q+1] \leftarrow \texttt{col}[q], \quad \texttt{val}[q+1] \leftarrow \texttt{val}[q]$
      $q \leftarrow q - 1$
    end while
    $\texttt{col}[q+1] \leftarrow c, \quad \texttt{val}[q+1] \leftarrow v$
  end for
end for

Insertion sort is chosen because rows of sparse matrices are usually short; for long rows a comparison sort such as quicksort on the pair $(\texttt{col}, \texttt{val})$ is preferable.

3 Complexity

Step	Time	Extra memory
1. Count	$O(M + \texttt{nnz})$	–
2. Prefix sum	$O(M)$	–
3. Cursors	$O(M)$	M integers
4. Scatter	$O(\texttt{nnz})$	–
Total (Alg. 1)	$O(M + \texttt{nnz})$	M integers
5. Sort rows (Alg. 2)	$O\left(\sum_i n_i^2\right)$ worst case	–

Here n_i is the number of nonzeros in row i. Since every entry must be read at least once, $O(M + \texttt{nnz})$ is optimal for the conversion itself.

4 Worked example

$$A = \begin{pmatrix} 10 & 0 & 1 & 7 \\ 0 & 3 & 3 & 0 \\ 0 & 21 & 0 & 0 \\ 4 & 0 & 5 & 6 \end{pmatrix}, \qquad M = N = 4,\ \texttt{nnz} = 9.$$

COO input (file order, 0-based):

k	0	1	2	3	4	5	6	7	8
R	3	0	2	1	0	3	1	0	3
C	2	0	1	2	3	0	1	2	3
V	5	10	21	3	7	4	3	1	6

- After Step 1: `row_ptr` $= [0, 3, 2, 1, 3]$.
- After Step 2: `row_ptr` $= [0, 3, 5, 6, 9]$.
- After Step 4: `val` $= [10, 7, 1 \mid 3, 3 \mid 21 \mid 5, 4, 6]$, `col` $= [0, 3, 2 \mid 2, 1 \mid 1 \mid 2, 0, 3]$.
- After Step 5: `val` $= [10, 1, 7, 3, 3, 21, 4, 5, 6]$, `col` $= [0, 2, 3, 1, 2, 1, 0, 2, 3]$.